

**BABEȘ-BOLYAI UNIVERSITY CLUJ-NAPOCA
FACULTY OF MATHEMATICS AND COMPUTER
SCIENCE**

SPECIALIZATION Sisteme informatice avansate

DISSERTATION THESIS

A Cloud-Based Closed-Loop Payment System with Hybrid On-Chip Reconciliation for Festival Environments

**Supervisor
Assoc. Prof. Christian Sacarea, PhD**

*Author
Tudor Esan*

2026

UNIVERSITATEA BABEȘ-BOLYAI CLUJ-NAPOCA
FACULTATEA DE MATEMATICĂ ȘI INFORMATICĂ
SPECIALIZAREA Sisteme informatice avansate

LUCRARE DE DISERTAȚIE

**Un sistem de plată closed-loop în
cloud cu reconciliere hibridă pe cip
pentru festivaluri**

Conducător științific
Assoc. Prof. Christian Sacarea, PhD

Absolvent
Tudor Esan

ABSTRACT

This thesis describes a cloud-based closed-loop payment system for music festivals and large outdoor events. The system is online by default and uses the bracelet balance as a fallback when the network goes down. Festivals are known for bad connectivity when a lot of people gather in the same place, but in our view, building the whole system around the assumption that the network will fail brings more complexity than it is worth. We keep things online most of the time using a Starlink uplink and a redundant network topology, and we let the NFC wristband hold the balance as a cache for the rest. When the network is up, transactions go through the server in real time. When it is not, the chip itself stores the balance and a monotonic debit counter, so vendor terminals can approve a payment locally and not let anyone spend money twice. As soon as connectivity comes back, the terminals push their queued transactions to the server in batches, and the server applies them against its own credit counter. Credits are written only by the server and debits are written only by the chip, so the two flows meet again on their own when the network is back. We assess the architecture against what a real festival actually needs and we argue that treating offline as a bounded fallback, and not as the foundation of the system, gives a simpler and more maintainable result.

Contents

1	Introduction	1
1.1	Problem Statement	2
1.2	Objectives	4
1.3	Thesis structure	5
2	Background and Related Work	6
2.1	Closed-Loop Payment Systems	6
2.1.1	Transit Card Systems	7
2.1.2	Festival Payment Systems	7
2.2	Offline authorization in open-loop systems	8
2.3	Offline-First Architecture Patterns	8
2.3.1	Local-First Software	9
2.3.2	Conflict-Free Replicated Data Types	9
2.3.3	Event Sourcing and CQRS	9
2.3.4	Edge Computing	10
2.4	Conflict Resolution in Distributed Systems	10
2.4.1	The CAP Theorem	10
2.4.2	Eventual Consistency	10
2.4.3	Transaction Ordering	10
3	System Architecture	12
3.1	High-level overview	12
3.1.1	The hybrid reconciliation model	13
3.2	Balance reconciliation model	13
3.2.1	State on each side	14
3.2.2	Invariants	14
3.2.3	The four operations	15
3.2.4	Worked example	15
3.2.5	Lost transactions	16
3.3	Lessons from related systems and patterns	16
3.3.1	Offline operation	17

3.3.2	Top-ups: the one thing that needs the network	17
3.4	The mobile application	17
3.4.1	Attendee view	17
3.4.2	Vendor POS mode	18
3.5	Admin dashboard	18
3.6	Backend server	19
3.6.1	Multi-tenancy	19
3.6.2	Horizontal scalability	20
3.7	Database	20
4	Implementation	21
4.1	Implementation overview	21
4.1.1	Project layout and libraries	21
4.1.2	Deployment	22
4.2	Database schema for the hybrid model	22
4.3	Authentication and bearer tokens	24
4.4	The reconciliation service and sync endpoint	24
4.5	Chip data layout, HMAC, and the key chain	26
4.6	Online debit path	27
4.7	Offline debit path and the queue	28
4.7.1	Auto-sync on reconnect	29
4.8	Top-up path and the Stripe webhook	29
4.9	Tests	30
5	Security Considerations	31
5.1	Threat model	31
5.2	Chip integrity and the per-bracelet key	31
5.3	Cryptographic key management	32
5.4	Server-side integrity	33
5.5	Authentication	34
5.6	Stripe and PCI scope	34
5.7	Vendor terminal trust	35
5.8	Data protection	35
5.9	Denial of service and availability	35
5.10	Residual risks	36
6	Discussion	37
6.1	Design Trade-offs	37
6.1.1	Online by default, offline as a fallback	37
6.1.2	Server balance versus chip balance	37

6.1.3	Consistency versus availability during outages	38
6.1.4	Lost transactions and operator risk	38
6.1.5	Starlink and a redundant topology versus other connectivity options	39
6.2	Comparison with Existing Solutions	39
6.2.1	Commercial festival platforms	39
6.2.2	Transit cards	40
6.2.3	Open versus proprietary	41
6.3	Limitations	41
6.3.1	Hardware: a plain NFC chip is not enough for production . .	41
6.3.2	Offline duration is assumed to be short	41
6.3.3	Single-event scope in testing	42
6.3.4	No peer-to-peer transfers between attendees	42
6.3.5	Refund flow is partial	42
6.3.6	No real load testing	42
6.3.7	Vendor onboarding is a manual review	42
6.4	Further Improvements	43
6.4.1	Move to authenticated NFC chips	43
6.4.2	Fraud detection on the server	43
6.4.3	Real-time analytics for the operator	43
6.4.4	Multi-event support and tenant isolation	43
6.4.5	Vendor settlement and payout automation	44
6.4.6	Integration with other festival systems	44
6.5	Evaluation against the objectives	44
7	Conclusions	46
7.1	Verdict per objective	46
7.2	Contributions	46
7.3	Closing thought	47
	Bibliography	48

Chapter 1

Introduction

Music festivals and large outdoor events have become a massive part of modern culture. Every summer, millions of people gather in open fields, parks, and temporary venues to enjoy live performances and food. From Untold in Romania to Tomorrowland in Belgium, these events keep growing in size and complexity. But behind all the excitement, there is a very practical problem that organizers have to deal with: how do you let tens of thousands of people buy food, drinks, and merchandise in a place where the internet barely works?

The traditional approach was straightforward: people brought cash, stood in line, and paid at the counter. It was slow, it was messy with always having change and there was always the risk of theft or losing your wallet in the crowd. Over the past decade, many festivals have moved towards cashless payment systems, typically based on RFID wristbands or prepaid cards. The idea is simple: you load money onto your wristband before or during the event, and then you just tap it at a vendor terminal to pay. Industry data from cashless technology providers suggests that RFID systems reduce transaction times by up to 80 percent compared to cash and that festival organizers see average revenue increases of around 22 percent after going cashless [Tap24]. These figures are consistent with the academic evidence: a recent meta-analysis across 71 studies found a statistically significant, small positive effect of cashless payments on consumer spending ($g = 0.135$, 95% CI [0.068; 0.203]), driven primarily by the reduced psychological friction of not physically handing over money [SBH24].

However, these systems come with a critical weakness: most of them depend on a reliable internet connection. And if there is one thing that festivals are known for, it is terrible connectivity. When tens of thousands of people gather in the same area, all using their phones simultaneously, the cellular network simply cannot handle the load. Shafiq et al. [SJL⁺13] conducted the first large-scale characterization of cellular network performance during crowded events using real-world traces from a tier-1 US network. Their findings are impressive: pre-connection failures increased

by 100 to 5,000 times compared to routine days, driven by users exhausting the limited bandwidth of the signaling channel when too many devices attempt to acquire radio resources simultaneously. This degradation extended as far as 10 miles around the event venue as attendees arrived and departed. Even for users who managed to connect, voice call failures increased by 7 to 30 times, packet, and round-trip times increased by up to 7 times. The performance spikes were particularly severe during intermissions, when large numbers of attendees simultaneously attempted to use their devices. Notably, the study found that even with portable base stations and free Wi-Fi offloading already deployed, these degradation levels persisted, highlighting a fundamental capacity limitation rather than a simple configuration problem.

This is where things get problematic for payment systems. If the payment terminal at a food stall needs to contact a remote server to authorize a transaction, and the network is down or painfully slow, the transaction fails. The attendee cannot buy food. The vendor loses a sale. Multiply this by tens of thousands of transactions per day, and you can see why this is a serious issue. The most famous example of this going wrong happened at the Download Festival in the UK in 2015, the first major UK festival to go completely cashless. The RFID system experienced technical failures that left attendees unable to purchase food or drinks, with many taking to social media to describe the system as useless. Although organizers claimed only around one percent of attendees were affected, the backlash was severe enough that the festival abandoned the cashless-only approach the following year [eFe16].

The core question this thesis tries to answer is: can we build a payment system that works reliably at festivals and that does not fall apart when the network has a bad moment? Our approach is to design a system that operates online by default, with connectivity coming from a Starlink uplink and a redundant network topology, so that vendor terminals can talk to the cloud backend in real time most of the time. But no network is perfectly reliable, especially one assembled in a field for a few days, so we also need a fallback for the moments when the network is down. Instead of building a separate offline server or a heavy synchronization layer, we let the NFC wristband itself act as the cache, holding the balance and a small counter that the vendor terminal can update locally until the network comes back.

1.1 Problem Statement

Festivals put together a few problems that, on their own, payment systems already know how to solve, but putting them all together is when it becomes tricky. The first one is connectivity. Cellular networks fall over when too many people gather in the same place, and many festivals happen in fields and forests where there is no proper

wired infrastructure to begin with. Our answer is to bring our own connectivity to the venue: a Starlink uplink with a redundant network topology, so that the vendor terminals can talk to the cloud backend most of the time. When the network is up, transactions are authorized in real time, wallet balances are fresh, and operations that need an external provider like Stripe work as you would expect.

A network we set up ourselves is still a network, and it can still go down. Equipment fails, the density of wireless devices in a small area creates dead zones, and multi-day events keep stressing infrastructure that was put together in a field a few days before. When the network does drop for a few minutes during a peak hour, the system still has to take payments, otherwise people cannot buy a sandwich and the whole festival economy stops. So we need a fallback that takes over when the network is gone and gets out of the way when it comes back. We do not want to design the whole system around the assumption that the network will fail, because that would push complexity everywhere for a problem that is online most of the time.

The second piece is that this is a closed economy. Unlike a normal shop where you tap a bank card and the bank takes care of authorization, a festival has its own internal currency: attendees load tokens, spend them at approved vendors, and get refunded for what they did not use. The money never leaves the perimeter of the event. This is actually an advantage for us, because we control the whole flow and we are not blocked by an external authorization network that we could not reach during an outage anyway.

The third piece is scale and speed. A medium festival can have hundreds of vendors processing thousands of transactions per hour, with very sharp peaks around meals and between concerts. Nobody is going to wait two minutes for a payment to go through. Online, this is just a matter of provisioning the backend to handle the spike. Offline, the question becomes how to keep the same speed at each terminal without giving up correctness, which is the next problem.

The fourth piece is consistency during offline windows. If two terminals lose the network at the same time and both approve a payment from the same wallet, they could together let through more money than is actually there. This is the well-known double-spend problem, and it is pretty hard in distributed systems. Refusing the payment is not an option for us, because at a festival “we cannot pay because the Wi-Fi is bad” is not an acceptable answer. So we choose availability, and we make sure the architecture limits how badly things can go wrong: each wristband carries its own balance and a debit counter on the chip itself, so the chip is the one that decides locally whether a payment fits, and double-spend across terminals is not possible because the chip is a single physical object that the customer carries between vendors. If minor discrepancies arise the festival will cover the loss, and the

worst case per terminal can be kept small by limiting the size of the offline queue and the maximum amount of a single transaction, both of which are knobs the operator can set ahead of the event.

The fifth piece is reconciliation. When terminals come back online, the transactions they processed offline have to land in the server in a way that does not lose anything and does not double-apply anything. Because we are online-first, these windows are expected to be short and not that frequent, but even a few minutes of outage during a peak can produce hundreds of queued transactions per terminal, and the sync has to be idempotent and order-tolerant.

One last thing worth saying out loud: not every operation can work offline. Wallet top-ups in particular need real-time communication with Stripe (or any other on-ramp provider) and with our backend, because real money is moving from the attendee's bank account into the system. You cannot process that on the chip. This is on purpose: the chip handles debits when the network is gone, but credits, refunds, and anything that touches external money stay strictly online and stay strictly on the server.

1.2 Objectives

The main objective of this thesis is to design and implement a minimum viable product (MVP) for a cloud-based, closed-loop payment system tailored to festival and event environments. The system is online by default, with connectivity coming from a Starlink uplink and a redundant network topology, and uses the NFC wristband itself as a local cache so that vendor terminals can keep approving payments during short network outages. The concept of a minimum viable product, as defined by Ries [Rie11], refers to a version of a product that allows the team to collect the maximum amount of validated learning with the least effort. In our case, this means building enough of the system to demonstrate that the core architectural ideas work, without necessarily implementing every feature a production deployment would need. Throughout this work, we explore different design architectures and patterns, discussing their benefits and trade-offs in the context of festival environments, before settling on the approach that best fits the constraints of the problem.

Specifically, the objectives of this work are the following:

O1: Design a payment architecture that keeps a closed-loop wallet usable through short network outages, without splitting the codebase into separate online and offline implementations. The same vendor-side flow has to work whether the network is up or down, with the chip and the server taking turns as the local source of truth depending on what is reachable.

O2: Build the wallet service that supports the two operations a festival needs in practice: top-ups through an external payment provider while online, since real money is entering the system, and debits at vendor terminals both online and during outages, with the terminal queueing offline debits for a later batch sync.

O3: Specify a reconciliation model that converges back to a consistent balance after offline windows without ad-hoc conflict resolution, and verify it with end-to-end tests that simulate the cases the model has to handle (idempotent retries, contention between two terminals, top-ups that race with offline debits).

O4: Implement a core service that manages vendors, products, stock, users, and events, providing the administrative backbone of the system.

O5: Build the client applications attendees, vendors, and event organizers need: a single mobile app that adapts to the attendee or vendor role on the same device, plus a separate web dashboard for organizers to set up and run events.

O6: Discuss the trade-offs of the chosen architecture and verify it against the festival requirements outlined in Section 1.1, deferred to Chapter 6.

1.3 Thesis structure

The rest of the thesis follows the path from the prior art to the running system and back to a verdict. Chapter 2 reviews the existing work that informs the design: transit cards, commercial festival systems, EMV's offline authorisation, and the distributed systems literature on conflict resolution. Chapter 3 presents the system architecture, with the hybrid two-counter reconciliation model at its core. Chapter 4 walks through the parts of the running code that matter for that model: the database schema rework, the on-chip data layout, the sync endpoint, and the offline queue. Chapter 5 discusses the threat model and the load-bearing weakness around cryptographic key management in the current MVP. Chapter 6 returns to the trade-offs, compares the result against commercial systems, and evaluates the architecture against the objectives stated above. Chapter 7 summarises what the project did and did not achieve.

Chapter 2

Background and Related Work

This chapter reviews the prior art that informs our design. We start with closed-loop payment systems and the transit and festival deployments that have shaped the field, then look at EMV’s offline authorisation as the canonical open-loop counter-example, and finally summarise the architectural patterns and distributed-systems theory that the design choices in Chapter 3 rest on. The arguments *for* our specific architecture live in later chapters; here we describe what already exists.

2.1 Closed-Loop Payment Systems

A closed-loop payment system, as described by SDK.finance [SDK24], “confines transactions to a specific network or ecosystem.” The same operator is the issuer and the acquirer, so the money stays inside the operator’s perimeter from top-up to refund. This is the opposite of an open-loop system like Visa or Mastercard, where the transaction crosses banks, card schemes, and processors that the merchant does not control.

Festivals lean toward closed-loop for four reasons. **Connectivity.** A festival is already a place where the network has a bad time, so depending on external authorisation hosts on the other side of the world is asking for trouble. With a closed loop, all authorisation logic lives inside the operator’s own infrastructure. **Fees.** Open-loop processing eats interchange, scheme, and processor fees per transaction; in a closed loop, the only point where external fees apply is when money enters the system through a top-up, and operators usually negotiate lower rates with a strategic provider. **Regulation.** In the EU, electronic money is a regulated activity [Eur09], but festival systems often fit under the limited-network exemption of PSD2 [Eur15] by issuing prepaid tokens redeemable only inside the venue. Showing balances as “tokens” rather than euros reinforces this framing. **Control.** The operator decides transaction limits, refund policies, vendor approvals, and settlement timing without

a card scheme telling them what is allowed.

The one moment a closed-loop system has to talk to the outside world is when real money enters the system. This requires an active connection to an external provider like Stripe and is therefore tied to the network being up.

2.1.1 Transit Card Systems

Transit cards (Suica in Tokyo, Oyster in London, Octopus in Hong Kong) are the most successful example of closed-loop payments that work without constant connectivity. Shirakawa [Shi02] describes how Suica's FeliCa chip [Son24b] handles fare deduction locally at the gate through encrypted read-write operations, completing each transaction in roughly 0.1 seconds without a network round-trip. Mutual authentication between the chip and the gate generates a fresh session key per transaction, which keeps cloning hard: the chip's secret key never leaves its secure memory, so captured radio traffic is useless on the next tap. Newer FeliCa generations use AES and the most recent revision is certified at Common Criteria EAL6+, which covers resistance to physical and side-channel attacks. Logs are uploaded from gates to central servers in batches for reconciliation and reporting, so these systems are not purely offline; they are offline at the point of sale and online for the back office.

Octopus [Son24a, CP03] adopted FeliCa four years before Suica and grew from a transit card into a general-purpose stored-value card accepted at convenience stores, restaurants, and parking. By the early 2000s it had over 11 million cards in a city of 7 million, which shows that the closed-loop, card-stored-balance pattern scales beyond its original domain.

The relevant takeaways for a festival system are: sub-second transactions are a hard requirement when crowds queue, an on-chip balance is what makes that possible, and the controlled environment of a transit network (fixed terminals, private back-office network, predictable usage) does not transfer directly to a temporary venue with mobile vendor terminals.

2.1.2 Festival Payment Systems

Cashless festivals have been deployed by Tappit, Intellitix, Weezevent, and Glownet at hundreds of events worldwide. The typical setup hands out an RFID wristband at the gate, lets attendees load credits through cash stations or an online portal, and processes payments at vendor terminals that read the wristband, deduct the amount, and write the new balance back. Industry data [Tap24] reports notable speed and revenue gains versus cash, and academic work on cashless payment be-

haviour [SBH24] confirms a small but statistically significant positive effect of cashless on consumer spending.

Two limitations of the wristband-only model are widely documented. First, the chips most commonly used are MIFARE Classic, whose proprietary CRYPTO1 cipher was broken in practice by Garcia et al. [GdKGM⁺08]; more modern DESFire chips address this but are more expensive, so older deployments still rely on the weaker hardware. Second, the architectures of these commercial systems are proprietary and essentially undocumented in academic literature. The failures that are public, such as the Download Festival 2015 crash mentioned in Chapter 1, are reconstructed from press reports rather than from any published post-mortem.

Conversations with people involved in payment infrastructure at Untold (one of the largest festivals in Romania) added a piece of practical context: even with on-site servers and a private network, top-up sync delays of two to four hours across terminals were common when Wi-Fi coverage degraded. Reliable Wi-Fi at venue scale is itself a hard infrastructure problem, not just a deployment detail.

2.2 Offline authorization in open-loop systems

EMV (named after Europay, Mastercard, and Visa) is the canonical open-loop card standard. It is not closed-loop, but it is the most widely deployed example of offline authorisation, and it is worth understanding because it solves a similar problem under stricter constraints. EMV cards carry a set of risk parameters: a per-card offline floor, a maximum number of consecutive offline transactions, and a cumulative offline amount. When the merchant terminal cannot reach the issuer’s host, it authorises locally if all three checks pass and forces an online attempt otherwise.

The design distributes risk: the issuer decides how much offline exposure a given cardholder is allowed, and the merchant terminal can have its own floor below which it always accepts. The known weakness, demonstrated by Murdoch et al. [MDAB10] as a man-in-the-middle attack on EMV’s PIN verification, is the broader point that any system whose authorisation depends on a device in the user’s possession is vulnerable if that device is compromised.

2.3 Offline-First Architecture Patterns

Building a system that has to work without a network puts it somewhere on a spectrum from fully online (every operation needs a round-trip) to fully offline (the network is only used to sync logs in the background). Real systems sit in between. This section reviews four architectural patterns from the literature that are relevant to

systems on that spectrum.

2.3.1 Local-First Software

The term “local-first software” was introduced by Kleppmann et al. [KWvHM19]. The principle is that the primary copy of the data lives on the user’s device, and the cloud syncs between devices instead of being the source of truth. The seven ideals listed in the paper (no spinners, multi-device, optional network, seamless collaboration, longevity, security, ownership) define a target for collaboration apps.

The pattern fits document-style data, where two divergent versions can be merged at any time without losing meaning. It does not fit balances directly, because money is a conserved quantity rather than a piece of text: if two replicas of a balance both authorise a withdrawal, the merge cannot simply union them. The argument for treating local-first as a partial fit, as a read cache rather than as the primary copy, belongs in Chapter 3; here we record only what the principle says.

2.3.2 Conflict-Free Replicated Data Types

CRDTs, formalised by Shapiro et al. [SPBZ11], are data structures whose operations are commutative, associative, and idempotent, so replicas converge regardless of the order updates arrive in. The simplest example is a G-Counter (grow-only counter), and PN-Counters extend the idea to allow decrements through paired increments.

CRDTs guarantee convergence but not invariants. Kleppmann’s later work on JSON CRDTs [KB17] notes that the converged state is not necessarily one any single replica would have accepted on its own. For a payment system that has to enforce “balance ≥ 0 ”, this is the relevant limitation: the convergence story is solved, but the constraint enforcement is not.

2.3.3 Event Sourcing and CQRS

Event sourcing, as described by Fowler [Fow05], stores the sequence of events that produced the current state instead of the state itself. The current state is reconstructed by replaying events, which gives a complete audit trail by construction. CQRS (Command Query Responsibility Segregation), described by Young [You10], separates the write side of a system (commands that produce events) from the read side (queries against a denormalised projection). The two patterns are often used together, with events as the durable record and the read model derived from them.

2.3.4 Edge Computing

Edge computing, as framed by Shi et al. [SCZ⁺16], moves computation closer to where data is produced rather than centralising it. In a festival the equivalent is the vendor terminal: it can read the chip, validate, and record locally, only involving the cloud when the network is reachable. Trade-off: edge nodes work with potentially stale data; how stale is acceptable, and how to bound the resulting risk, is a design choice.

2.4 Conflict Resolution in Distributed Systems

When multiple terminals act independently they will end up with different views. Distributed systems have a long literature on what guarantees are achievable in this setting and what concepts are useful for thinking about it.

2.4.1 The CAP Theorem

Brewer’s conjecture, formalised by Gilbert and Lynch [GL02], says a distributed system cannot simultaneously guarantee consistency, availability, and partition tolerance: when a partition happens, the system has to pick one of the first two. Network partitions in a festival environment are a regular occurrence, not an edge case, so the partition leg of CAP is not optional.

2.4.2 Eventual Consistency

Vogels [Vog09] defines eventual consistency as the guarantee that, if no new updates are made, all replicas eventually converge. The word “eventually” is doing a lot of work; in practice convergence happens quickly once partitions heal. The canonical real-world example is Amazon’s Dynamo [DHJ⁺07], which accepts writes on any reachable node and resolves conflicting versions with application-specific logic when partitions heal.

2.4.3 Transaction Ordering

Distributed systems theory offers logical and vector clocks for tracking causal ordering between events on different nodes when physical clocks disagree [DHJ⁺07]. These tools are necessary when concurrent writers can target the same object and the system needs to know which write happened first. Whether they are needed in any given system depends on whether such concurrency exists, not on the absolute scale of the workload.

For festival transactions, the realistic granularity matters: transactions happen at human speed (a person walks to a vendor and taps), and even cheap NTP keeps wall-clocks within a few hundred milliseconds of each other, which is far below the spacing between two transactions involving the same wristband. Whether wall-clock timestamps are sufficient as an ordering mechanism in a specific architecture is a design question we revisit in Chapter 3.

Chapter 3

System Architecture

This chapter goes through the architecture of the closed-loop payment system. We start with the big picture, explain why we chose this approach, and then go into each component. The idea is to keep things simple: one server, one app for everyone, and reliable network whenever possible. If the network goes down, the balance cached on the bracelet is what decides whether a payment is approved.

3.1 High-level overview

Most existing festival payment systems need a technical team on-site to set up local servers and a private network for the duration of the event. As discussed in Section 2.1.2, festivals like Untold bring dedicated servers to the venue because the balance lives on a remote server and every transaction needs a network round-trip.

Our system takes a different path. We aim to always have reliable internet using Starlink as the primary uplink, paired with a redundant network topology. The wristband-as-cache fallback exists for residual edge cases, but the goal is to never need it. This means there is nothing to deploy at the venue: no local servers, no private network, no on-site IT team. The organizer creates an event from the admin dashboard, configures it, approves vendors, and distributes wristbands. The server runs in the cloud the same way it runs for every other festival on the platform.

The system has four main components: a backend server, a mobile application for attendees and vendors, a web-based admin dashboard, and NFC wristbands that carry the token balance for each attendee. The server is a single multi-tenant service that manages all festivals from one deployment. The mobile app is what attendees and vendors interact with: attendees see their balance and history, vendors use it as a soft point-of-sale terminal. Admins manage the event through a separate web dashboard. The wristband is where the balance actually lives during the event.

3.1.1 The hybrid reconciliation model

The core architectural decision is how the wristband and the server share responsibility for the balance. The state and the rules that keep the two sides convergent are described in detail in Section 3.2; the threat model around the chip and the upgrade path to authenticated chips are in Chapter 5.

High Level Flow

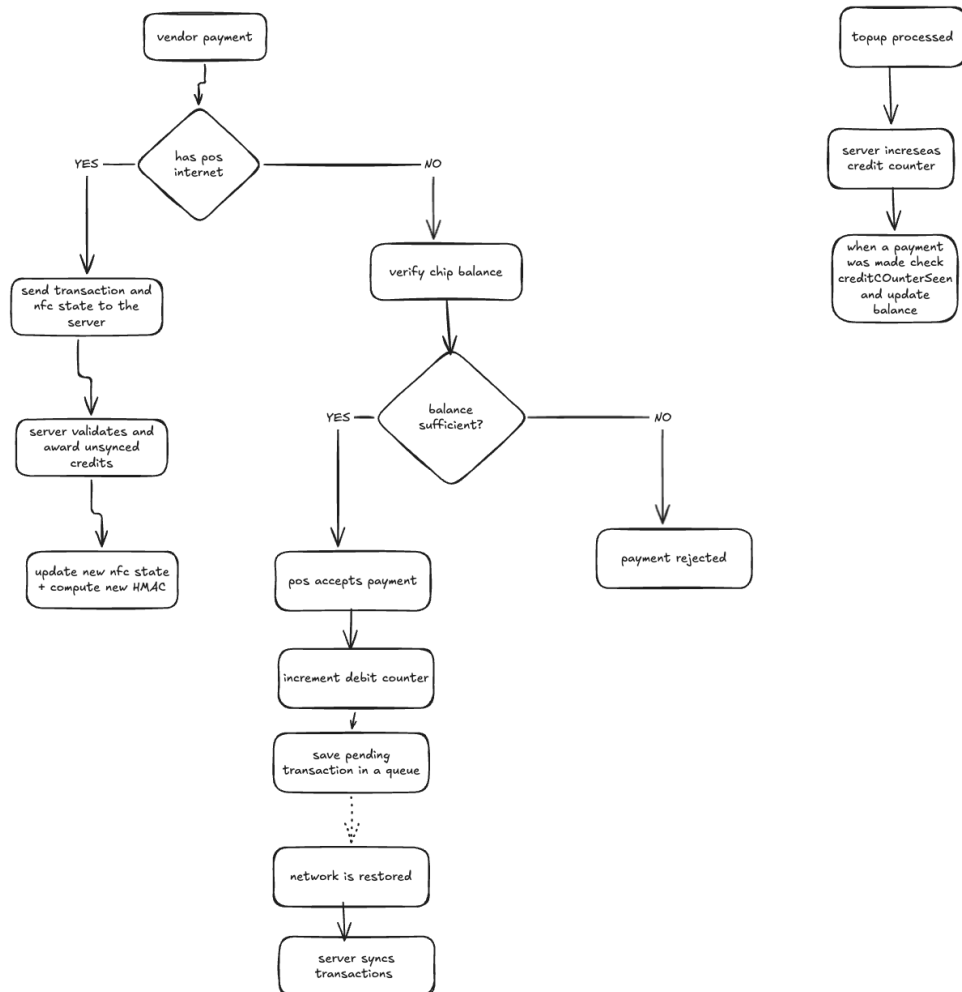


Figure 3.1: High-level system architecture. Debits happen directly between the vendor device and the NFC wristband. Credits go through the backend, which is also responsible for top-ups, sync, and reporting.

3.2 Balance reconciliation model

A closed-loop payment system with offline-capable terminals has two writers that can mutate the balance: the server (top-ups) and the chip (vendor payments). Dur-

ing a network partition both can act on their own, and once connectivity is back the system has to converge on a correct final value. Our hybrid model handles this by keeping the two writers in separate lanes that never overlap.

3.2.1 State on each side

The wristband stores three reconciliation fields per event, plus identity fields written at provisioning:

- `balance`: cached balance in minor units, what the chip can spend from when offline.
- `debit_counter`: monotonic counter, incremented by one on every chip-side debit. The chip is the only writer.
- `credit_counter_seen`: the highest server-side credit sequence number the chip has acknowledged.
- identity fields `bracelet_uid` and `event_id`, written once at provisioning.

The server stores the mirror image, per wristband per event:

- `balance`: the canonical balance.
- `credit_counter`: monotonic counter, incremented by one on every server-side credit. The server is the only writer.
- `debit_counter_seen`: the highest `debit_counter` value the server has reconciled from any terminal.

The two counters are independent monotonic sequences, each with one writer. Because each lane has exactly one writer, there is no merge conflict to resolve within a lane.

3.2.2 Invariants

The correctness of the model rests on three invariants:

1. **The chip never increases its own balance.** Every chip-side mutation is a debit: `balance -= amount` together with `debit_counter += 1`, applied as one atomic write.
2. **The server is the only source of credits.** Top-ups, refunds, and operator adjustments mutate the server's balance and increment `credit_counter`. The chip picks up these credits passively on the next tap by an online terminal.

3. **Counters are monotonic with a single owner each.** `debit_counter` only ever grows on the chip; `credit_counter` only ever grows on the server.

As long as these three hold, reconciliation reduces to taking the union of two non-conflicting streams.

3.2.3 The four operations

Top-up. Always online. The server checks the payment, increments `balance` and `credit_counter`, writes a transaction row, and acknowledges. The chip is not touched and gets reconciled on the next tap.

Online debit. The terminal reads the chip and forwards the request to the server with the chip's current `balance`, `debit_counter`, and `credit_counter_seen`. If the chip's debit counter is ahead of the server, the server trusts the chip as evidence of offline spending that has not synced yet. It reconstructs the spendable balance from the chip state plus any server credits the chip has not seen, caps that value by the server balance, validates the new debit, advances `debit_counter_seen`, and returns the new triple (`balance`, `debit_counter`, `credit_counter_seen`), which the terminal writes atomically to the chip.

Offline debit. The terminal reads the chip, checks the balance, and performs an atomic decrement on the chip. It generates an idempotency key and queues the transaction locally, draining the queue in the background as connectivity allows.

Sync. When a terminal regains connectivity, it pushes queued transactions to the server in batches. The server applies new counters idempotently using the per-transaction key, updates `balance`, and advances `debit_counter_seen` to the highest counter value observed. If a queued debit arrives later for a counter that the server has already absorbed through an online chip-aware debit, the server still records the transaction row for vendor history and settlement, but does not subtract the amount again.

3.2.4 Worked example

Consider a top-up, an offline debit, and then a later online purchase. Starting state: chip (`balance=50`, `debit_counter=0`, `credit_counter_seen=0`) and server (`balance=50`, `credit_counter=0`, `debit_counter_seen=0`).

The user pays 10 tokens at an offline terminal. The chip becomes (40, 1, 0) and the terminal queues a debit of 10 with counter 1. In parallel, a 20-token top-up goes through the server, which becomes (70, 1, 0).

The user then pays another 5 tokens at an online terminal. The terminal reads (40, 1, 0) and forwards that chip state with the new debit request. The server

sees that the chip is one debit ahead of `debit_counter_seen`, and that the chip has not yet seen credit counter 1. It therefore computes the spendable balance as $40 + 20 = 60$, capped by the server balance of 70. After the 5-token debit, the server stores `balance=55` and `debit_counter_seen=2`, and the terminal writes (55, 2, 1) to the chip.

When the offline terminal eventually syncs the queued debit for counter 1, the server does not subtract 10 again, because the online debit already absorbed that missing chip-side spend into the aggregate balance. It inserts the late debit as a historical transaction row for reporting and settlement, marks it as already absorbed, and leaves the balance at 55. Both sides converge without blocking the online purchase on the background sync timing of another vendor device.

3.2.5 Lost transactions

The model does not magically prevent loss. If a terminal dies before draining its queue, the vendor served the customer and the chip subtracted the amount, but the server may initially have no itemized transaction row. The online debit path still trusts the chip's lower balance, so the attendee cannot spend the same tokens twice. The missing itemized debit remains detectable as a server transaction-history gap for a counter value below `debit_counter_seen`.

The policy is to trust the chip's balance immediately for authorization, wait a configurable grace period for late syncs to fill in itemized vendor history, and after that log the missing counter range as a discrepancy. The festival absorbs any settlement loss for transactions whose terminal queue never returns; vendors with local receipts can claim manually. The customer balance exposure is bounded because online payments use the chip state when it is ahead of the server.

3.3 Lessons from related systems and patterns

The hybrid model borrows from three places: transit cards (Section 2.1.1) for the on-chip balance pattern, EMV (Section 2.2) for the idea of bounding offline risk through per-card limits, and the offline-first literature (Section 2.3) for what to take and what to leave. From transit we keep the chip-as-spend-cache idea, with credits handled by the back office instead of by the gate. From EMV we keep the framing that offline always trades availability for some risk that has to be capped explicitly. From the patterns in the literature, local-first [KWvHM19] fits as a read cache rather than as the primary copy because money is conserved; CRDTs [SPBZ11] are overkill once each balance lane has a single writer; event sourcing is used directly, since every transaction is an immutable row keyed by an idempotency key; and the vendor

terminals act as edge nodes [SCZ⁺16] that run the debit logic locally when the network is gone. The operational lesson from the Untold conversations is the one that pushed us off the on-site-server pattern: even with dedicated infrastructure at the venue, top-up sync delays of two to four hours were common when Wi-Fi degraded.

3.3.1 Offline operation

During online operation, vendor terminals send each payment request to the server together with the chip state read from the wristband. When the network goes down, the wristband interaction still succeeds locally: the terminal debits the chip and queues the transaction on the vendor device. Once connectivity returns, the device pushes its queued transactions in a background batch. The full convergence mechanics are covered in Section 3.2.

3.3.2 Top-ups: the one thing that needs the network

Topping up is the one operation that cannot happen offline. Real money moves from the attendee's bank account through Stripe into the operator's account, which requires an internet connection. Restricting top-ups to online-only is also what keeps the credit lane fully under server control.

Top-ups happen in two ways. Attendees can top up through the app using Stripe; once the payment clears, the server records the new balance and increments `credit_counter`, and the next vendor tap picks up the pending credit. Cash top-up stations at the event submit the credit through the server, which then writes the new balance to the wristband on the spot.

3.4 The mobile application

There is one mobile app, built in React Native, that attendees and vendors both download. What the user sees depends on their role: attendees see their balance, vendors get a POS terminal. We went with React Native because the same codebase needs to run on regular phones (attendees) and on devices used as POS terminals (vendors) that require NFC support.

3.4.1 Attendee view

Attendees see a simple screen showing their token balance, a transaction history, and a way to top up using Stripe. The balance shown in the app is the last known state synced from the server; the real balance during the event lives on the wristband. The app is mostly informational for attendees.

3.4.2 Vendor POS mode

When an admin has whitelisted a device, the app switches to POS mode on that device. The vendor enters the amount and the customer taps their wristband. If online, the app reads the chip state, sends it to the server with the amount, and writes the server's returned state back to the chip. If offline, the app checks the chip balance locally, deducts the amount, increments the chip debit counter, writes the new state back, and stores the debit in the local queue for background sync.

To become a vendor, the business provides basic information (business name, contact, products). The admin reviews and approves the application, then whitelists specific devices (by device ID) as POS terminals. This whitelisting matters because not just any phone can start accepting payments.

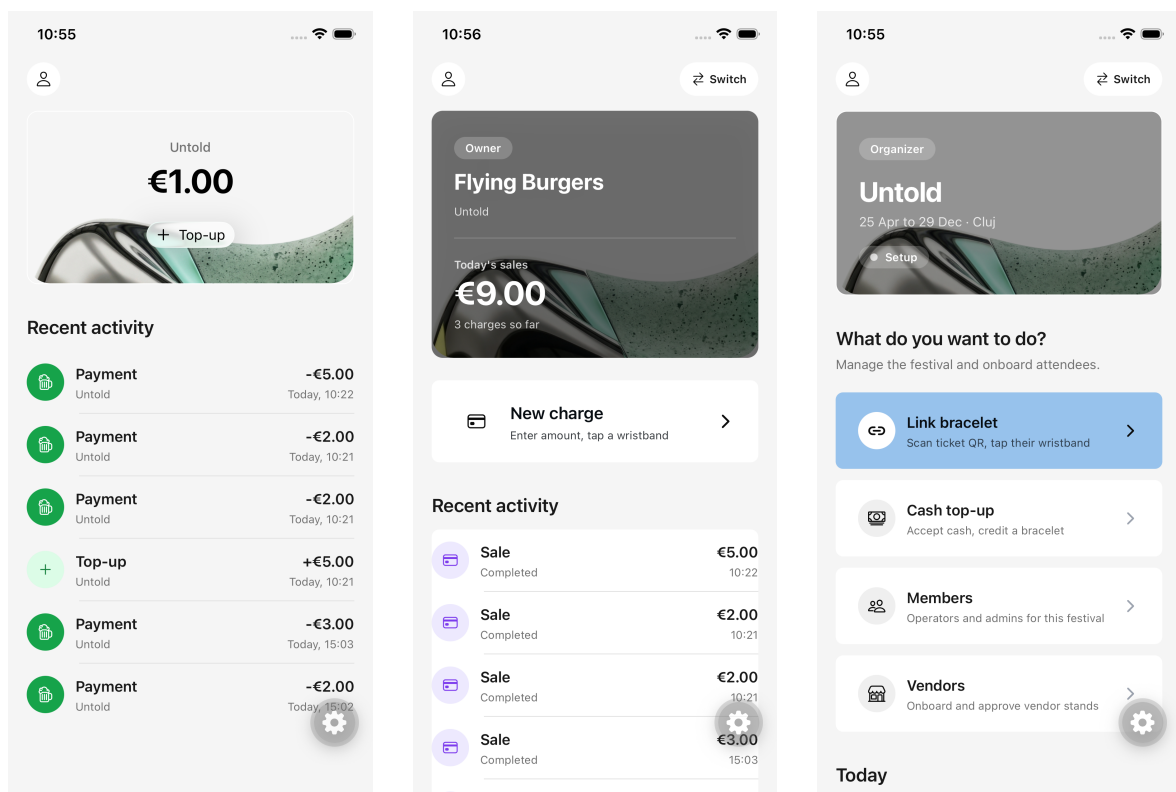


Figure 3.2: Role-specific mobile screens in the shared React Native application. The attendee view shows personal balance and spending history, the vendor view turns the phone into a soft POS terminal, and the mobile admin view gives operators quick access to event controls.

3.5 Admin dashboard

The admin dashboard is a separate web application built in React. Making it a web-site means any organizer can access it from any browser without downloading anything. The dashboard covers four areas:

Event setup. Create the event, set the token-to-currency rate, define spending limits, configure vendor commissions, and set dates. The event has a lifecycle (setup, active, settlement, closed) and the dashboard controls the transitions.

Vendor management. Review vendor applications, approve or reject them

Monitoring and statistics. Real-time transaction volume and other stats

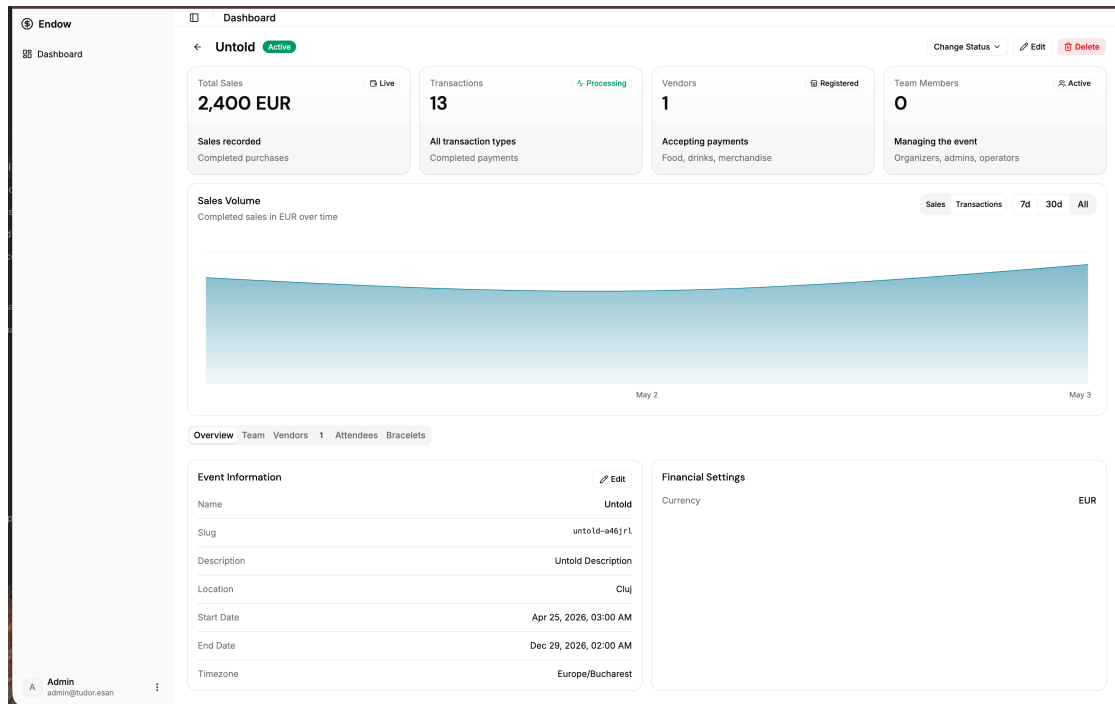


Figure 3.3: Web admin portal for festival operators. The portal gives organizers a browser-based control surface for event configuration, vendor review, payment monitoring, and operational statistics.

3.6 Backend server

The backend is a single multi-tenant service. It exposes four families of endpoints (authentication and accounts, event management, vendor management, and sync/reconciliation), is stateless across instances, and runs behind a managed load balancer. The choice of framework, the database, and the deployment target are implementation details that belong to the next chapter (Section 4.1.2).

3.6.1 Multi-tenancy

The server handles all festivals from a single deployment. Every table that belongs to an event is reachable from the event row, either directly through an `event_id` foreign key or through `event_bracelets`. API calls include the event context (path

parameter or token claim) and queries are filtered accordingly. Adding a new festival is just creating a row in the events table.

3.6.2 Horizontal scalability

The server is stateless: no session data or cached state in memory. Every request gets what it needs from the database and from the bearer token sent by the client. The service can run as many instances as needed behind the load balancer with no sticky sessions or inter-instance communication. For most festivals a single instance is enough, since the majority of payments happen on the wristband without touching the server in real time.

3.7 Database

The data model follows the hybrid reconciliation model directly. The canonical balance and the two reconciliation counters live on `event_bracelets`, every money movement is logged in `transactions` with the appropriate counter set, and top-up records sit in `payment_intents`. The full schema (table list, column purposes, constraints, and the safety nets that Postgres enforces directly) is described in Chapter 4, Section 4.2.

The choice of a single relational database rather than a sharded or distributed setup is conservative on purpose. Most transactions never touch the database in real time, because they happen on the wristband during normal vendor flow; the database mostly absorbs sync batches and dashboard queries. A recent published reference point [Ope26] from OpenAI describes ChatGPT’s user-facing workload, with hundreds of millions of users, running on a single primary Postgres with read replicas at low double-digit millisecond p99 latency, which suggests that a festival’s volume is well inside what a single relational database handles. If load ever justifies it, the standard scaling path (read replicas, connection pooling, table partitioning) does not require changes to application code.

Chapter 4

Implementation

This chapter describes how the architecture from Chapter 3 was turned into running code. The focus is on the parts that are specific to the hybrid balance model: the database schema rework, the on-chip data layout, the sync endpoint, and the reconciliation logic that ties everything together. Topics like screen design, navigation, or generic CRUD endpoints are mentioned only briefly because they do not say much about the system as a whole.

4.1 Implementation overview

The system is built on three codebases that live in the same monorepo. The backend is a NestJS server in `backend/`, talking to a Postgres database through Drizzle ORM. The vendor terminal is a React Native application built with Expo, in `softpos/`, using `react-native-nfc-manager` for chip reads and writes. The thesis itself lives in `docs/`.

The whole stack uses TypeScript end to end. The data shapes that travel between the chip, the terminal, and the server are easy to get wrong, and a single set of types shared between client and server catches most of those mistakes at build time. Drizzle’s schema definitions also act as the source of truth for the database, so the table layout and the application types stay in sync without a separate code generation step.

4.1.1 Project layout and libraries

The backend is a standard NestJS 11 project: controllers, services, modules, with class-validator on the DTOs and Drizzle for the database. Authentication uses Better Auth with the `bearer` and `emailOTP` plugins; there is no custom JWT layer. Stripe integration goes through the official `stripe` SDK. Postgres access is through the `postgres` driver.

The terminal is an Expo app. NFC is handled by `react-native-nfc-manager`. The offline queue is stored in plain JSON files via `expo-file-system`. Connectivity changes are observed through `@react-native-community/netinfo`. Cryptography on the chip side (HKDF for key derivation, HMAC-SHA256 for chip integrity) uses `@noble/hashes`. Bearer tokens for the backend are kept in `expo-secure-store`, since React Native does not have a cookie jar. HTTP requests go through `axios` and server state is cached with `@tanstack/react-query`.

4.1.2 Deployment

The backend runs on DigitalOcean's App Platform, behind a managed load balancer. The database is a serverless Neon Postgres instance in a European region, accessed through `@neondatabase/serverless` on the backend side. The choice of region is driven by the latency profile we want for festivals in Romania and the rest of Europe.

4.2 Database schema for the hybrid model

The hybrid model from Chapter 3 required a real cleanup of the schema. The original design had a generic `wallets` table where each user owned a single balance, shared across events. That made sense when the system was thought of as a closed-loop wallet platform, but it does not match how festivals actually work. An attendee at one festival does not expect leftover credit to appear at a different festival run by a different organizer, and an organizer wants their event balances to be isolated for accounting and refunds. Balance is therefore event-scoped, and the natural place for it is the row that already represents the link between an attendee and a specific event.

That row is `event_bracelets`, and it now carries the balance directly. Three new columns were added: `balance` (an integer count of cents, defaulting to zero), `debit_counter_seen` (the highest `debit_counter` value the server has applied from the chip for this bracelet), and `credit_counter` (a server-owned counter that increases by one every time the server credits the bracelet, for example after a successful top-up). The two counters are independent on purpose. Debits are allocated by the chip when a payment happens at a vendor, and credits are allocated by the server when money flows in. Keeping them separate means neither side has to coordinate with the other to pick a fresh value.

The `transactions` table was reshaped to match. The legacy `transaction_counter` column was dropped because it tried to merge debits and credits into one sequence, which made it impossible to tell which side a row was authored by. In its place, two

nullable columns were added: `debit_counter`, set on debit rows, and `credit_counter`, set on credit rows. Exactly one of the two is populated for any given transaction. The foreign key `wallet_id` was retargeted to `event_bracelet_id`, and the redundant `event_id` column was removed because the event is reachable through the bracelet. The same retargeting was applied to `payment_intents.wallet_id`.

Two tables were dropped entirely. The `wallets` table is no longer needed because the balance moved to `event_bracelets`, and `vendor_invitations` was removed because the invitation flow was replaced by a simpler model where adding a user as a vendor creates the user account on the spot if they do not already have one. The remaining membership tables (`event_tickets`, `event_members`, `vendor_members`) were kept as is.

Table	Column	Purpose
<code>event_bracelets</code>	<code>balance</code>	Current balance in cents, event scoped
<code>event_bracelets</code>	<code>debit_counter_seen</code>	Last debit counter applied from chip
<code>event_bracelets</code>	<code>credit_counter</code>	Server-owned credit counter
<code>transactions</code>	<code>debit_counter</code>	Set on debit rows, nullable
<code>transactions</code>	<code>credit_counter</code>	Set on credit rows, nullable
<code>transactions</code>	<code>event_bracelet_id</code>	Replaces <code>wallet_id</code>
<code>payment_intents</code>	<code>event_bracelet_id</code>	Replaces <code>wallet_id</code>

Table 4.1: New and changed columns introduced by the hybrid model rework.

The schema also relies on Postgres for two safety nets that the application code does not have to repeat. A check constraint on `event_bracelets` requires `balance >= 0`, so any path that would push a balance negative aborts the transaction at the database layer. Two partial unique indexes scoped on `status = 'active'` make sure that, for any given event, there is at most one active assignment per wristband UID and at most one active assignment per user. These two indexes are partial on purpose: revoked or replaced rows stay around for audit, but they do not block re-issuing a bracelet to the same person.

The end result is a schema that maps cleanly onto how operators think about a festival. A bracelet belongs to an event, the balance lives on that bracelet, and every movement of money against it is logged in `transactions` with a counter that says whether it was a debit (allocated by the chip) or a credit (allocated by the server). The full table list is: `users`, `sessions`, `accounts`, `verifications` (Better Auth), `events`, `event_members`, `vendor_members`, `vendors`, `event_bracelets`, `event_tickets`, `transactions`, `payment_intents`, and `audit_logs`. Every table that belongs to an event is reachable from the event row, either through an `event_id` foreign key or through `event_bracelets`, which gives the multi-tenant boundary without a tenant column on every table.

4.3 Authentication and bearer tokens

Authentication is handled by Better Auth on both sides. The backend declares the auth instance in `src/common/auth/auth.ts`, with the `expo`, `bearer`, and `emailOTP` plugins. There is no custom JWT layer: the server issues an opaque session and the mobile app sends the session token as `Authorization: Bearer <token>`. The token itself is kept in `expo-secure-store`, since React Native does not have a cookie jar to lean on the way a web browser does. Login goes through the email-OTP flow: the user types their email, the backend sends a six-digit code, the app posts the code back, and Better Auth returns the token. Authorization for vendor and admin operations is done through the `event_members` and `vendor_members` tables rather than a global role on the user row, so the same person can be an attendee at one event and a vendor operator at another without a role change at the user level.

4.4 The reconciliation service and sync endpoint

The terminal sends the queued debits it has for one bracelet, and the server returns the outcome the terminal needs to clean up its local queue.

```
POST /api/v1/events/:eventId/wristbands/:wristbandUid/sync
Authorization: Bearer <token>
Content-Type: application/json
```

Request:

```
{
  "pendingDebits": [
    {
      "idempotencyKey": "<uuid>",
      "amount": <int cents>,
      "vendorId": "<uuid>",
      "counterValue": <int>,
      "deviceId": "<string>",
      "clientTimestamp": "<ISO 8601>"
    }
  ]
}
```

Response 200:

```
{
  "serverState": {
    "balance": <int cents>,
    "debit_counter_seen": <int>,
    "credit_counter": <int>
  },
  "applied": ["<idempotencyKey>", ...],
  "rejected": [
    { "idempotencyKey": "...",
      "reason": "duplicate" | "insufficient_funds" }
  ],
  "chipShouldWrite": {
    "balance": <int cents>,
    "credit_counter": <int>
  }
}
```

The terminal does not send the chip's HMAC, because by the time `pendingDebits` reaches the server those debits have already been written to the chip and the HMAC has been recomputed. What matters at sync time is whether each entry in the queue has actually landed on the server's ledger.

The reconciliation algorithm runs inside a serializable Postgres transaction (`SET TRANSACTION ISOLATION LEVEL SERIALIZABLE`) that locks the bracelet row for the duration of the call. Serializable isolation matters here: two terminals that both held the same bracelet during overlapping offline windows can hit the sync endpoint at the same time, and the database has to make their effects look like a strict ordering. With a weaker level a race between two reconciliations could let the same chip counter be applied twice or could allow the balance to go negative.

1. Open a serializable transaction and lock the bracelet row.
2. Sort `pendingDebits` by `counterValue` ascending.
3. For each pending debit:
 - a. If `idempotencyKey` already exists in transactions, record it as `duplicate` and continue.
 - b. If `counterValue` $\leq$ `bracelet.debit_counter_seen`, insert a historical debit if that counter is not already present, without changing `bracelet.balance`, then continue.
 - c. If `bracelet.balance` $<$ `amount`, record it as `insufficient_funds` and continue.

- d. Decrement `bracelet.balance` by `amount`.
- e. Set `bracelet.debit_counter_seen = counterValue`.
- f. Insert a transaction row with `debit_counter = counterValue` and the supplied `idempotencyKey`.
4. Read the current `bracelet.balance` and `bracelet.credit_counter`.
5. Build `chipShouldWrite` from that current state.
6. Commit the transaction.
7. Return `serverState`, `applied`, `rejected`, and `chipShouldWrite`.

The two rejection reasons cover the cases the service produces today. The `duplicate` reason fires when the same debit reaches the server twice, either because the terminal retried after a flaky response or because a transaction row for the same bracelet counter already exists. The `insufficient_funds` reason fires when the queued debit is ahead of the server state and the server cannot honour it from the current balance. A late debit whose counter is already behind `debit_counter_seen` is not rejected automatically: if no row exists for that counter, the service records it as historical itemized vendor data and marks the metadata as already absorbed. The `RejectReason` type leaves room for two more categories (`counter_gap` for non-contiguous counters and `invalid` for malformed entries) but the current service does not produce them; they are reserved for future hardening.

The response still carries `chipShouldWrite` for observability and parity with the online charge response, but background sync does not write to the chip. Chip writes happen only during live NFC interactions, especially online charge, where the bracelet is present and the server can compare its state with the database.

4.5 Chip data layout, HMAC, and the key chain

The chips used for the bracelets are NTAG class tags, which are cheap and widely available, but do not have the cryptographic features of DESFire. They cannot do mutual authentication on their own, so the security has to come from a signed record written into the user-memory pages. The on-chip layout is small and fixed.

<code>balance</code>	<code>uint32 little-endian (cents)</code>
<code>debit_counter</code>	<code>uint32 little-endian</code>
<code>credit_counter_seen</code>	<code>uint32 little-endian</code>
<code>hmac</code>	<code>bytes(16) HMAC-SHA256 truncated to 128 bits</code> <code>over the 12 bytes above</code>

The first twelve bytes are the state. The last sixteen bytes are an HMAC computed over those twelve bytes using a per-bracelet symmetric key. Anyone reading

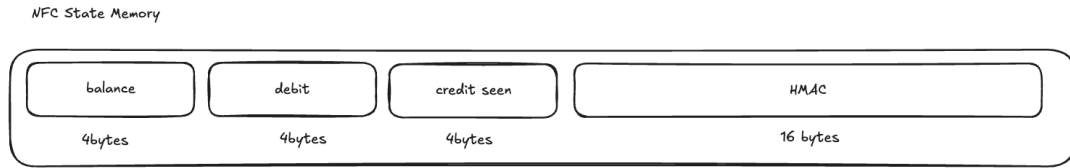


Figure 4.1: NFC user-memory record used by the prototype. The first twelve bytes carry the mutable balance state and counters; the trailing sixteen bytes store the truncated HMAC that protects the record against unsigned edits.

the chip can read the state, but only someone who knows the per-bracelet key can produce a record that the terminal will accept.

The per-bracelet key is derived rather than stored. The terminal holds a master key and computes the bracelet’s working key on the fly with HKDF-SHA256:

```
key = HKDF(masterKey, salt = wristbandUid, info = "bracelet-v1")
```

The wristband’s UID is read from the chip’s read-only manufacturer area, so it cannot be spoofed by simply rewriting the user memory. The `info` string `"bracelet-v1"` fences the derivation against reuse if a future version of the layout ever needs to co-exist with this one.

In the current MVP the master key sits on the terminal: it is read from the `EXPO_PUBLIC_BRACELET_MASTER_KEY` environment variable at build time, with a hex fallback baked into the source so a fresh checkout still runs. Expo inlines `EXPO_PUBLIC_*` variables into the JavaScript bundle, so anyone with the app binary can extract the master key and derive every per-bracelet key. This is acceptable for a thesis demo but it is not a production posture; Chapter 5 discusses the trust implications and the upgrade path (per-event keys provisioned through an HSM, or moving to a DESFire chip that holds its own key).

4.6 Online debit path

When a vendor terminal has connectivity, the simplest and safest thing to do is to push the debit to the server first and let the chip catch up afterwards. The terminal posts the debit to a charge endpoint that takes the bracelet, the amount, the vendor identity, and an idempotency key generated on the device. The server opens a serializable transaction on the bracelet row, checks that the balance covers the amount, decrements the balance, allocates the next `debit_counter` value (one above whatever was last applied), inserts a `transactions` row with that counter, and returns the new balance and `debit_counter` to the terminal.

The terminal then writes the new state back to the chip in a single NFC session, recomputing the HMAC over the new balance, debit counter, and the unchanged `credit_counter_seen`. If that write succeeds, the transaction is closed. If it fails because the customer pulled their wrist away too early, the server-side debit has already happened and the chip is now stale relative to the ledger. The next time the chip is read by any terminal, the server's view will be lower than the chip's view, and on the next sync the terminal can write the corrected state down. This is the same self-healing path the offline flow relies on, just hit from a different angle.

The decision to do server-first online and chip-first offline is intentional. When the network is up, the server is the cheapest place to enforce invariants and the chip is the more failure-prone half. When the network is down, the chip is the only place where the invariant can be enforced at all. The model swaps which side leads based on what is actually available.

4.7 Offline debit path and the queue

The offline path starts on the terminal. When connectivity is missing, or when the request to the charge endpoint times out, the terminal falls back to local authorisation. It reads the chip, derives the per-bracelet key from the bundled master key with HKDF, verifies the HMAC, and checks that the balance covers the amount.

Before touching the chip, the offline branch enforces two limits that bound how much a single terminal can accumulate while disconnected. The terminal refuses new offline payments once the local queue already holds `OFFLINE_QUEUE_CAP` pending debits (50 in the current build), and it refuses any single offline payment above `OFFLINE_MAX_AMOUNT_CENTS` (100.00 in the current build). Both checks run before the NFC write, so a refused payment never decrements the bracelet; the operator is told to sync the terminal or to take the payment online. The product of the two constants is the most a terminal that dies before syncing can cost the festival, which is the bound the risk discussion in Section 6.1.4 builds on. The two values are compile-time constants for now; a production deployment would provision them per event together with the rest of the terminal configuration.

If the check passes, the terminal performs the debit on the chip inside a single NFC session. The new balance is computed (`balance - amount`), the debit counter is incremented by one, the credit counter seen is left untouched, the HMAC is recomputed over the three new fields, and the resulting twenty-eight bytes are written back to the chip. The blob spans seven NTAG pages, which the transport layer writes one page at a time (NTAG WRITE is a per-page command). The multi-page write is therefore not atomic on its own. What makes the chip-side update behave atomically as far as the rest of the system is concerned is the HMAC: a torn

write leaves the body bytes inconsistent with the trailing HMAC, and the next read fails verification before any logic acts on the partial state. The terminal is then expected to retry the write or fall back to the queue.

To stop concurrent NFC operations from stepping on each other, all chip reads and writes go through a single `withSession()` helper that chains them through one promise, so that the higher level code can call `readChip` and `writeChip` from anywhere without having to think about overlapping sessions.

Only once the chip write is acknowledged does the terminal enqueue the pending debit on disk. The queue is stored as plain JSON files under the app's document directory, one file per (event, vendor) pair, written through `expo-file-system`. Each entry carries the same fields the sync request will eventually need: a generated `idempotencyKey`, the amount, the `vendorId`, the `counterValue` that was just written to the chip, the `deviceId`, and the local timestamp. Storing the entry only after the chip write means the queue can never contain a debit that did not happen; the worst case is a chip update that the terminal forgot, which the next reconciliation will detect through the counter.

When connectivity comes back, the terminal posts the queue to the sync endpoint described in Section 4.4. The server replies with which entries it accepted and which it rejected. The terminal removes the accepted entries from the JSON file. Rejected entries are kept and surfaced to the operator with the reason: `duplicate` is silently dropped, and `insufficient_funds` is escalated to a manual review screen because it implies the queued debit and the server balance disagreed in a way the automatic logic should not paper over.

4.7.1 Auto-sync on reconnect

The terminal does not wait for the operator to re-tap a bracelet before flushing the queue. A subscription to `@react-native-community/netinfo` watches the connectivity state, and on a transition from offline to online it triggers a queue drain across every (event, vendor) pair the terminal has seen during the session. The drain only uploads queued debits; it does not require the bracelet to be present and does not send a stored chip snapshot. A subsequent online charge tap is responsible for bringing the chip back to the server-returned state.

4.8 Top-up path and the Stripe webhook

Top-ups are the credit side of the system, and they always go through the server because they involve real money moving through Stripe. The terminal (or the attendee's own phone, depending on the flow) creates a Stripe Payment Intent against

the backend, which returns the client secret. The user confirms the payment in the Stripe SDK. On success, Stripe sends a `payment_intent.succeeded` webhook to the backend.

The backend has `rawBody` enabled at the application level, so the webhook controller can read `req.rawBody` and pass it together with the `Stripe-Signature` header to the Stripe SDK for signature verification. Verifying against the raw body, before any JSON parser has touched it, is the only way the signature will check out; this is why a body parser cannot be turned on globally.

Once the signature is valid, the handler looks up the corresponding `payment_intents` row by the intent ID and runs the credit inside a Postgres transaction. The transaction does not set an isolation level, so it runs at the default `read committed` level. The handler increments `bracelet.balance` by the topped-up amount, increments `bracelet.credit_counter` by one, and inserts a `transactions` row with that new `credit_counter` value and a reference to the payment intent.

Idempotency for top-ups does not come from a unique key on the credit transaction. Instead, two layers protect against duplicate credits. The `payment_intents` table has a `UNIQUE` constraint on `provider_intent_id`, so two webhooks for the same Stripe intent cannot create two rows. On top of that, the handler short-circuits if the intent row is already in status `succeeded`: a webhook retry from Stripe finds the intent already marked as completed, returns immediately, and does not run the credit a second time. The combination is enough to make the webhook handler safe to retry, which Stripe relies on aggressively.

After the credit is committed, the next online charge tap returns a `chipShouldWrite` that reflects the new balance and the new `credit_counter`. The terminal writes this back to the chip, and the chip's `credit_counter_seen` catches up to the server's `credit_counter`. From that point on the chip and the server agree, and the bracelet is ready to be debited again, online or offline.

4.9 Tests

The backend ships with three test suites. `reconciliation.e2e-spec.ts` drives the sync endpoint against a real Postgres started inside a `Testcontainers` container, exercising idempotency, counter checks, balance underflow, and the worked example from Section 3.2.4. `bracelets.e2e-spec.ts` covers bracelet assignment, replacement, and the issuance of the short-lived bracelet token used by the terminal during a session, against the same kind of containerised database. `bracelet-token.spec.ts` is a unit test that exercises the token signing, verification, and expiry logic without touching the database.

Chapter 5

Security Considerations

A festival closed-loop payment system has a smaller attack surface than an open-loop one (the money never leaves the operator's perimeter, the merchants are vetted, and the cards are short-lived) but it inherits a familiar set of problems: someone may try to tamper with a chip, a vendor terminal can be lost or rooted, and the webhook surface has to assume an attacker can speak to it. This chapter describes the threats the implementation actually addresses today, the residual ones, and the upgrade path for a production deployment.

5.1 Threat model

The assets worth attacking are the bracelet balance, the credit lane (top-ups), and the vendor settlement totals. The actors we worry about are an attendee with an off-the-shelf NFC writer, a malicious or compromised vendor with physical access to a terminal, an attacker on the public network speaking to the backend, and Stripe acting as a trusted external party. The trust boundaries are: between the chip and the terminal (the chip holds no secret of its own and is read by anyone with NFC); between the terminal and the backend (an authenticated TLS channel); between the backend and Stripe (signed webhooks and authenticated API calls); and inside the backend, between the public HTTP layer and the database (a private connection through `@neondatabase/serverless`).

5.2 Chip integrity and the per-bracelet key

The bracelets in the current implementation are plain NTAG class chips. This is an important limitation, not a production security choice. NTAG tags expose simple read and write memory pages and do not perform mutual authentication with the terminal. They also do not keep a secret on the chip. As a result, the system cannot

prove that a physical bracelet is the original bracelet rather than a copied tag with the same user-memory contents.

The protection added by the prototype is therefore not anti-cloning. It is integrity protection for the balance record stored in NTAG user memory. Every chip blob is twelve bytes of state followed by a sixteen-byte HMAC-SHA256 truncated to 128 bits, computed with a per-bracelet key derived through HKDF from a master key kept on the terminal (Section 4.5). A direct rewrite of the user pages with a fabricated balance fails verification on the next read because the attacker does not have the per-bracelet key needed to recompute the HMAC. In practical terms, the HMAC prevents someone with a basic NFC writer from changing `balance`, `debit_counter`, or `credit_counter_seen` to arbitrary values and having the terminal accept them.

The counters add replay resistance at reconciliation time, but they should not be confused with chip authentication. If an old signed record is written back onto the tag, the server can detect the stale `debit_counter` or `credit_counter_seen` when the bracelet later meets an online terminal. If a whole NTAG is cloned, however, both copies initially carry a valid signed state. The backend can reject duplicate counter values once the conflicting debits reach the server, but the clone may still be usable during the offline window. What bounds this exposure is the balance carried on the cloned chip (a clone cannot spend more than the balance it was copied with) together with keeping offline windows short so the duplicate counters surface quickly (Section 6.3.2). The NTAG hardware itself contributes nothing to this bound.

For a production deployment, the chip should be upgraded to a secure NFC product such as MIFARE DESFire EV2 or EV3, or an equivalent chip with diversified keys, encrypted storage, and mutual authentication. In that design, the chip can prove possession of a secret during the transaction and the terminal does not merely trust a signed blob copied from memory. The current HMAC layout is useful for demonstrating the offline balance model, but it is not a substitute for secure-chip authentication.

What this construction also does not defend against is an attacker who learns the per-bracelet key. Since the key is derived deterministically from the master key and the wristband UID, recovering the master key is enough to forge any chip on the festival.

5.3 Cryptographic key management

This is the load-bearing weakness of the current MVP. The master key lives on the terminal. It is read from the `EXPO_PUBLIC_BRACELET_MASTER_KEY` environment variable at build time, and Expo inlines `EXPO_PUBLIC_*` variables straight into the

JavaScript bundle. Anyone who pulls the APK from a vendor phone can extract the bundle, find the hex string, and compute the per-bracelet key for every wristband at the festival. That removes the chip-level HMAC defence in one move.

There is a second related issue. The source has a fallback master key (a hex constant) that activates if the environment variable is missing at build time. The fallback exists so that a fresh checkout of the repository runs without any setup, which is convenient during development but dangerous in production: a build that forgets to set the env var ships a known key. Either removing the fallback or making the build fail loudly when the env var is unset is a hard requirement before any production use.

The honest framing is that the chip-side cryptography buys time but not strong assurance, as long as the master key sits in the mobile bundle and the wristband is a plain NTAG tag. Two upgrade paths close different parts of the gap. The cleanest production path is to move to a chip with its own secure element, such as DESFire EV2 or EV3, and AES-128 mutual authentication; the chip can hold diversified keys and resist memory cloning. The interim path, if NTAG hardware has to stay for a pilot, is per-event keys provisioned to the terminal at event start through a short-lived authenticated channel, so that compromising one APK only compromises one event and only for the duration of that event. This interim path reduces key blast radius but still does not make NTAG unclonable. Either way, the hybrid reconciliation model on the server side is unaffected.

5.4 Server-side integrity

On the server side the invariants are easier to enforce because the database is a real source of truth. Reconciliation and online charge both run inside transactions with `SET TRANSACTION ISOLATION LEVEL SERIALIZABLE`, which closes the race where two terminals could otherwise apply the same chip counter twice or push the balance below zero between the read and the write. Top-ups run at the default `read committed` level because the credit path has only one writer per intent and idempotency is enforced at a higher layer (Section 4.8).

The schema also pins two invariants directly in Postgres. The `event_bracelets.balance >= 0` CHECK constraint makes any code path that would push a balance negative fail at the database; this is a backstop in case application-level checks ever drift. Two partial unique indexes on `event_bracelets` (scoped to `status = 'active'`) prevent two simultaneous active assignments per (event, wristband) and per (event, user), so a forgotten revoke cannot allow a duplicate active bracelet for the same person.

The single-writer-per-lane rule from Chapter 3 is structural: no application path

increments `credit_counter` from the chip side, and no application path increments `debit_counter_seen` ahead of a real chip-supplied value. As long as that rule holds, reconciliation reduces to merging two non-overlapping monotonic streams.

5.5 Authentication

Better Auth handles user sessions on both backend and terminal (Section 4.3). Authentication is by email OTP, so there is no password to leak; the OTP is one-shot and the bearer token is scoped to a single device. The mobile app stores the token in `expo-secure-store`, which is backed by the OS keystore on iOS and Android, so a non-rooted device does not expose the token to other apps.

Authority for vendor and admin actions is held in `event_members` and `vendor_members` rather than a global role column on `users`. This is on purpose: the same person can be an attendee at one festival and a vendor operator at another, and the per-event row is the right place to express that. The gap to be aware of is that the role guard library does not by itself check the event scope: any role check that needs to be event-bound has to fetch the membership row explicitly. This is a discipline issue rather than a framework hole, and it is one of the things a security review before deployment should sweep through.

5.6 Stripe and PCI scope

Card data never reaches the backend. The mobile app uses the Stripe SDK, which collects card details directly into Stripe's iframe and returns a tokenised intent that the backend creates and confirms through the Stripe API. This keeps the application out of PCI scope: the backend stores the `provider_intent_id`, the amount, and the user it belongs to, but no PAN, CVC, or expiry.

The webhook surface is locked down in two ways. First, the controller verifies the `Stripe-Signature` header against the raw request body (`rawBody: true` at the application level), so a forged webhook from somewhere else is rejected before it touches business logic. Second, the handler short-circuits if the intent is already in status `succeeded`, so a replayed webhook does not double-credit a bracelet. The `UNIQUE` constraint on `payment_intents.provider_intent_id` backs this up at the database level.

5.7 Vendor terminal trust

A vendor terminal is a phone that an admin has whitelisted by device ID. The whitelist is the difference between a phone that can take payments and one that cannot. This is enough to keep an attendee phone from masquerading as a vendor terminal, but it does not protect against a vendor whose phone was rooted between the time it was whitelisted and the time it took payments. A rooted device exposes both the bearer token and the master key in the bundle, so the operational mitigation is to keep the offline window short, refresh tokens regularly, and revoke devices the moment a vendor reports a lost or stolen phone. The longer-term fix is the same as for the chip: a hardware-backed key store for the master key, or a chip that holds its own.

5.8 Data protection

Personal data on the system is limited to what is needed to run the event: an email address for login, the wristband UID, and a transaction history scoped to that wristband. There is no name, address, or payment-method data on the application database; card data lives at Stripe. Traffic is TLS end to end, both between the terminal and the backend (HTTPS to the App Platform load balancer) and between the backend and the database (the `@neondatabase/serverless` driver enforces TLS).

For GDPR purposes the operator is the controller and the platform is the processor. The retention model that fits the data is short: transactions are kept for the legally required accounting window and then aggregated; the wristband-to-user mapping can be dropped at the end of the event without touching the financial records. Right-to-erasure requests resolve cleanly because the only durable PII handle is the email on the user row.

5.9 Denial of service and availability

The biggest availability concern at a festival is not a DDoS aimed at the backend but a flood of legitimate traffic during a network blip. The two-counter model is the structural answer: when the backend is unreachable the terminals fall back to chip-side authorisation and queue the rest, so a reachability problem at the load balancer does not stop people from buying food. On the backend itself, rate limiting at the platform level catches the obvious cases (a single client hitting an endpoint thousands of times a second) and the database connection pool caps per-instance load.

5.10 Residual risks

Three risks remain visible in the current shape of the system. The master key sitting in the mobile bundle is the largest one, and the security of every per-bracelet derivation depends on it. Plain NTAG chips can be cloned because the chip does not authenticate itself; the HMAC only proves that the stored balance record was produced by someone with the key. The window of fraud per clone is bounded but non-zero (one offline session worth of debits before the counters disagree). The vendor terminal has no platform attestation, so a rooted device cannot be told apart from an honest one without out-of-band signals.

None of these are fatal for the architectural model. The hybrid reconciliation logic does not change between the MVP posture and a production posture. The chapters that follow discuss the operational and product gaps that are also worth closing before a real deployment.

Chapter 6

Discussion

This chapter looks back at the system we built and tries to answer the questions that were left open in the objectives. We talk about the trade-offs we made, how the system compares to what already exists on the market, what the current MVP does not cover, and what would need to happen for a real production deployment.

6.1 Design Trade-offs

6.1.1 Online by default, offline as a fallback

The most consequential decision was framing offline as a bounded fallback rather than the foundation of the architecture. After running the system through its tests, the operational point that justifies this choice is that the offline path almost never fires in normal flow: when the network is up, the server validates and the chip catches up; the offline branch only activates during the actual outage. Building the whole system around the assumption that every operation might happen offline would have forced every component to deal with queues, conflicts, and retries for a path that runs during a small fraction of total transactions. The cost is a reduced feature set during outages (no top-ups, no real-time analytics, no refunds), which is acceptable because the only operation that has to keep working is the chip-side debit, and that is exactly what the fallback does.

6.1.2 Server balance versus chip balance

There were two extremes to choose from. A single authoritative balance on the server with the chip as a pure cache would have been simpler when the network was up, but as soon as the network is gone the cache is all the terminal can read, and without a counter on the chip there is no way to detect double-spend across two terminals during the outage. A balance held only on the chip, transit-style, sidesteps

that problem at the cost of moving credit handling onto a physical machine, which does not match how festival top-ups work (Stripe in the app, no chip nearby). The two-counter model is the compromise: server-owned credits, chip-owned debits, single writer per lane. The angle that ch4 could not yet make is that this split also dictates the failure modes you can recover from automatically (lane convergence) and the ones you cannot (lost terminals, see below).

6.1.3 Consistency versus availability during outages

CAP forces a choice during a partition; the festival forces availability (Section 2.4.1). What the implementation made concrete is that the protection against double-spend during a partition does not come from any distributed consensus, but from the chip being a single physical object that the customer carries: only one terminal can talk to it at a time, and the atomic chip-side decrement closes the rest of the gap. The cost is that the server's view of the balance lags during an outage and that a terminal which dies before draining its queue loses transactions, which the next subsection picks up.

6.1.4 Lost transactions and operator risk

The reconciliation model is correct as long as terminals eventually sync. If a terminal dies before it can push its queue, the chip subtracted the amount but the server has no record. The gap is detectable from the counter, but the transaction details are gone with the device. It is worth being precise about what kind of loss this is. By the time a debit sits in the queue, the chip was already decremented and the customer walked away with the goods, and the festival already holds that customer's money from the top-up. So a lost queue does not let anyone create money or take money from an attendee; what it destroys is the itemized record that says which vendor made those sales, so the festival cannot settle them from its own books. The loss is an accounting one, bounded by the real sales made at that one terminal during the outage. The design keeps this worst case sizeable in advance: cap the queue length and the per-transaction amount, and the product of the two is the most a single dead terminal can cost. The terminal enforces both limits in the offline charge path (Section 4.7): it stops taking offline payments once 50 debits are pending and refuses any single offline payment above 100.00, and both checks run before the chip write so a refused payment never touches the bracelet. With these values the worst case per terminal is 5000.00, a number the operator can size to whatever loss the festival is willing to absorb. The limits are constants in the current build; provisioning them per event is left as production work.

6.1.5 Starlink and a redundant topology versus other connectivity options

We chose Starlink as the primary uplink because it gives us a pipe that does not depend on the local cellular network being able to handle a sudden density of devices. The Shafiq et al. [SJL⁺13] study is a good reminder of how badly cellular networks fall over when too many people gather in the same place. Starlink does not have this problem because the bandwidth is shared across a satellite footprint, not a single tower in a field. However it can be influenced by bad weather

The alternatives we considered were a private LTE deployment, an on-site mesh network between vendor terminals, or just relying on the venue's existing Wi-Fi. Private LTE is expensive and requires spectrum that is not always easy to get on short notice. A mesh network between phones is interesting in theory but the moment one device has the only path to the gateway, you get hot spots and weird routing problems. Venue Wi-Fi is rarely good enough for thousands of concurrent connections. Starlink with a redundant topology is not free either, but it is the most predictable option and it does not require the venue to have any infrastructure.

6.2 Comparison with Existing Solutions

6.2.1 Commercial festival platforms

Tappit, Intellitix, Weezevent, and Glownet have been deploying cashless systems for years. The architecture is mostly the same across them: RFID wristband, on-site server, vendor terminals on a private network, and a back office that handles top-ups and reporting. They do this at a much bigger scale than what we built, with reliability that has been proven across hundreds of festivals.

Where our approach is different:

- We do not require an on-site server or an IT team at the venue. The backend is a single multi-tenant cloud service. Conversations about the Untold deployment confirmed that even with on-site infrastructure, top-up sync delays of two to four hours were common during the event.
- We treat the wristband as a cache, not a passive token. The balance and the debit counter live on the chip, so the terminal does not need to talk to a local server to authorize a payment.
- The architecture is documented and the code is open. The commercial systems are essentially black boxes in the academic literature, and the failures we know

about (Download 2015, the Danish Nets outage in 2024) are reconstructed from press coverage and tweets.

Where the commercial systems are clearly ahead:

- **Hardware.** They use proper RFID wristbands with a secure element (DESFire EV2 or EV3), authenticated reads and writes, and tamper detection. Our MVP uses a plain NTAG-class chip without a secure element, which is fine for a thesis demo but not for production.
- **Operations.** They have on-site teams that can swap a broken reader in minutes, handle vendor onboarding at the gate, and deal with the long tail of weird customer issues. We do not have any of this.
- **Scale.** They have processed tens of millions of transactions across hundreds of events. We have a working prototype that has been tested in a development environment.

The honest summary is that we built a different architectural shape, not a better product. The commercial systems are the result of years of iteration on real festivals, and any open alternative would need a similar amount of time in the field to catch up on the operational side.

6.2.2 Transit cards

Transit cards (Suica, Octopus, Oyster) are the closest existing system to what we built, in the sense that they also store a balance on a chip and let a terminal make a local decision. The big difference is the credit flow: at a transit station, the credit also happens at a physical machine that writes to the card, so the card is the source of truth for both directions. In a festival the credit happens through Stripe in the app, with no chip nearby, so the server has to be the source of truth for credits.

The other difference is the threat model. Transit fraud at the per-transaction level is small (a few euros), and the operators have built their cryptographic protocols around this. Festival transactions can be larger (a meal, a round of drinks for a group), so the financial incentive to tamper with a chip is bigger, and a production deployment would need a chip with a secure element. The lesson we take from transit is that the on-chip balance pattern works at scale, but the security properties have to be sized for the value that can be moved.

6.2.3 Open versus proprietary

The main contribution of this thesis is an open, documented architecture for a closed-loop festival payment system. The commercial systems work, but they do not help anyone who wants to understand how to build one, or who wants to deploy a similar system without paying license fees and signing a multi-year contract. Smaller festivals in particular cannot afford the commercial platforms, and they end up either staying on cash or going with a half-broken homegrown solution.

We hope that having an architecture written down, with the trade-offs discussed and the failure modes acknowledged, makes it easier for the next person to build on top of it without repeating the same mistakes.

6.3 Limitations

6.3.1 Hardware: a plain NFC chip is not enough for production

The biggest limitation of the current MVP is that the wristband is a plain NTAG-class NFC chip without a secure element. The hybrid model itself does not depend on chip-side cryptography, but the security properties of the system do. A motivated attacker with a cheap NFC writer can read the balance, write a higher balance, and walk back to the vendor. The server can detect this on the next sync (counters do not line up), but by then the customer has already left with the food. Section 5 in the security chapter lays out the upgrade path to a DESFire EV2 or EV3 chip with AES-128 mutual authentication, which is what a production deployment should use.

6.3.2 Offline duration is assumed to be short

The reconciliation model assumes that outages are short, on the order of minutes, not hours. The maximum amount that can be lost from a dead terminal scales with how long the terminal has been offline, and during a multi-hour outage a single terminal could buffer hundreds of transactions before its battery dies. The terminal does enforce a hard cap on the queue (it refuses new offline payments once 50 debits are pending, Section 4.7), so the buffering is bounded. What is missing is an escalation path for prolonged outages: once a terminal hits the cap it simply refuses offline payments, and there is no operator flow for prioritising a sync of that terminal, raising the limit on the spot, or moving the vendor to another device. The cap is also a compile-time constant, so adjusting it per event means rebuilding the app, which a production deployment would replace with per-event provisioning.

6.3.3 Single-event scope in testing

The architecture is multi-tenant by design (events are scoped through the `event_id` chain), but we only ran end-to-end tests against a single event at a time. We did not test what happens when two events are running simultaneously and share the same backend. The schema should handle it, but the operational side (load isolation, audit log scaling, admin dashboard performance) was not exercised.

6.3.4 No peer-to-peer transfers between attendees

The system does not support transfers between attendees. If two friends want to split a meal, one of them has to pay and the other has to send money back through some other channel. We could add it as an online-only operation through the app, but it adds a fraud surface (free money transfer through fake accounts) that we did not want to deal with for the MVP.

6.3.5 Refund flow is partial

Refunds for unspent balance at the end of an event are something every commercial system handles, and we only have a basic version of it. The server can credit the balance back to the original Stripe payment method through the standard Stripe refund API, but the policy questions (refund window, partial refunds, fees, refund of cash top-ups) are not implemented and are largely operational decisions that a real festival would have to make.

6.3.6 No real load testing

We argued in Section 3.7 that PostgreSQL on Neon should handle a festival comfortably, citing the OpenAI [Ope26] numbers. We did not actually run a load test that simulates a realistic festival traffic pattern (sharp peaks at meals and between concerts, thousands of concurrent terminals). The numbers we have are good in theory, but a production deployment should run a load test against a realistic synthetic workload before the first event.

6.3.7 Vendor onboarding is a manual review

The current onboarding flow is that a vendor submits an application from the app, the admin reviews it manually, and approves or rejects it. This works for a festival with a couple of dozen vendors, but it does not scale to a venue with hundreds. A production system would need an automated vetting flow with KYC, document verification, and probably a tiered approval process.

6.4 Further Improvements

6.4.1 Move to authenticated NFC chips

The first thing a production deployment should do is move from plain NTAG chips to DESFire EV2 or EV3 with AES-128 mutual authentication. This closes the obvious tampering attack and brings the security level up to what transit systems and commercial festival platforms already have. The hybrid reconciliation model does not change at all, only the read and write paths on the terminal side need to do the cryptographic handshake.

6.4.2 Fraud detection on the server

Even with authenticated chips, server-side anomaly detection would catch things that the chip cannot see on its own. Examples: a wristband that suddenly debits at two terminals on opposite sides of the venue within a minute (cloning), a vendor whose ratio of offline to online debits is much higher than the average (possible terminal manipulation), or a sudden spike in average transaction amount on a specific terminal. None of this exists in the MVP and all of it would be valuable in production.

6.4.3 Real-time analytics for the operator

The admin dashboard currently shows transaction volume and basic stats, but it does not have the kind of real-time analytics that festival operators want during the event (top-selling vendors, queue lengths approximated by debit rate, top-up rate per gate, anomaly alerts). A production deployment would need a streaming analytics pipeline (probably something like ClickHouse or Materialize on top of the transaction log) to give operators the visibility they need to make decisions during the event.

6.4.4 Multi-event support and tenant isolation

The schema is multi-tenant but we did not test what happens when two big events run on the same deployment at the same time. Going to production would mean exercising this path: separate audit log streams per event, per-event rate limiting, per-event database quotas, and probably a way to spin up dedicated read replicas for very large events without changing application code.

6.4.5 Vendor settlement and payout automation

At the end of an event, vendors expect to get paid based on their sales minus the festival commission. The current system records all transactions with the vendor and commission attached, but the actual payout (computing the totals, deducting fees, sending money to the vendor's bank account through Stripe Connect) is a manual operation. Automating this with Stripe Connect and a settlement run that the admin can review and confirm would close one of the biggest operational gaps.

6.4.6 Integration with other festival systems

The wristband is already a piece of hardware that the attendee carries, and it could be used for more than payments: access control at gates, age verification at bars, VIP area access, queue-skip features. The data model already has the wristband as a first-class entity, and adding a permissions layer per wristband would let the same hardware drive several festival systems from a single source of truth. This is what some commercial platforms already do, and it is a natural extension of the current architecture.

6.5 Evaluation against the objectives

The objectives stated in Section 1.2 are evaluated below against the running system. The verdicts are deliberately qualitative: the architecture was tested through end-to-end suites against a real database, but no festival-scale load test was run, so the parts that depend on volume (latency at peak, sync throughput under thousands of concurrent terminals) are flagged as not measured rather than presented with invented numbers.

O1 (unified online/offline architecture). Met. The same vendor-side flow handles both cases: when the network is reachable the terminal posts to the charge endpoint, when it is not the terminal authorises against the chip locally and queues the result for sync. The reconciliation endpoint described in Section 4.4 is the single convergence point, and the same code path picks up queued debits whether the terminal regained connectivity through a manual sync or through the auto-sync triggered by `netinfo`.

O2 (wallet service for top-ups and debits). Met for the operations the system is meant to support. Stripe top-ups are wired through the webhook handler, including signature verification against the raw body and the idempotency layer that combines `provider_intent_id` uniqueness with the `succeeded` short-circuit. On-line and offline debits are both implemented and exercised through the test suite.

Refunds for unspent balance at the end of an event are partial, as Section 6.3.5 acknowledges.

O3 (reconciliation model with verification). Met. The two-counter model is implemented with serializable Postgres transactions on the reconcile and online-charge paths. The behaviour is exercised by `reconciliation.e2e-spec.ts` against a real Postgres started inside Testcontainers, covering idempotent retries, counter regression on duplicate scans, balance underflow rejection, and the worked example with a top-up racing an offline debit. The `event.bracelets.balance >= 0` CHECK constraint is the database-level backstop.

O4 (core service: events, vendors, users). Met as scaffolding. Events, vendors, products, users, and the per-event membership tables are in place; the bracelet tokenisation flow and the admin endpoints are usable end-to-end. Vendor onboarding remains a manual review (Section 6.3.7), which is fine for a single-festival MVP and not enough for larger venues.

O5 (client applications). Partially met, with the objective itself revised. The original O5 promised three apps; the actual build is one mobile application that switches between attendee and vendor roles based on device whitelisting, plus a separate web admin dashboard. This is two clients, not three. The decision to merge attendee and vendor into one binary was driven by the simpler distribution story (one APK to install, one update channel) and the observation that on the same device a person is rarely both at the same time. The objective in Chapter 1 has been rewritten to match what was actually built.

O6 (evaluation against festival requirements). The five requirements identified in Section 1.1 map to the system as follows. *Connectivity*: addressed structurally through Starlink as the primary uplink and the chip-side fallback for outages, but the actual outage tolerance was not tested at festival scale. *Closed economy*: implemented through the closed-loop wallet tied to the event scope, with money entering only through Stripe and never leaving the operator's perimeter during a transaction. *Scale and speed*: handled architecturally (stateless backend, single-row contention per bracelet, most transactions never touching the database in real time) and supported by the published reference points cited in Section 3.7, but no peak-TPS measurement was made on this implementation. *Consistency during offline windows*: the test suite covers the core cases (idempotent retries, counter regression, balance underflow, top-up racing an offline debit), and the lane-separation invariant is structural rather than something the application has to enforce explicitly. *Reconciliation*: idempotent and order-tolerant, covered by the same test suite. The honest summary is that the architecture meets the qualitative requirements; quantitative validation at festival scale is part of what a real deployment would have to add, and Section 6.3.6 flags this explicitly.

Chapter 7

Conclusions

This thesis set out to build a closed-loop payment system that stays usable at a festival when the network has a bad moment, without paying the engineering cost of a fully offline architecture. The result is a documented system with a hybrid two-counter reconciliation model at its core, an end-to-end TypeScript stack, and a working reference implementation that has been exercised against a real database through automated tests.

7.1 Verdict per objective

The verdicts below are the short version of Section 6.5. **O1** (unified online/offline architecture) is met: the same vendor flow handles both cases, with a single sync endpoint as the convergence point. **O2** (wallet service) is met for the supported operations; the refund flow is partial and is acknowledged as such. **O3** (reconciliation model with verification) is met, with serializable transactions on the reconcile and online-charge paths and the model exercised by `reconciliation.e2e-spec.ts` against a real Postgres. **O4** (core service) is met as scaffolding; vendor onboarding remains a manual review. **O5** (client applications) is met against the revised objective: one mobile app that role-switches between attendee and vendor, plus a separate web admin dashboard, instead of the three apps the objective originally listed. **O6** (evaluation) is delivered qualitatively in Section 6.5; the parts that depend on volume (peak TPS, multi-event load isolation, sustained sync throughput) were not measured and are flagged as future work in Section 6.3.

7.2 Contributions

The contributions of this work are best read as a documented architecture rather than a new theoretical result. They are:

- A hybrid two-counter reconciliation model that splits writers cleanly between the chip (debits) and the server (credits), so the two streams converge without ad-hoc conflict resolution. The model is described in detail in Section 3.2.
- A working reference implementation: a NestJS backend with Drizzle and Postgres, a React Native vendor terminal with HKDF-derived per-bracelet keys and HMAC-stamped chip blobs, an Expo-based offline queue stored on the device’s filesystem, and an end-to-end test suite covering the core reconciliation cases.
- An open and documented account of the design choices, including the ones we deliberately did not make. Local-first as the foundation, CRDTs as the convergence mechanism, an on-site server tied to the venue’s Wi-Fi, and a mesh network across vendor terminals were each considered and explicitly rejected for reasons that the thesis records.
- An honest threat model around the cryptographic key chain (Chapter 5), including the load-bearing weakness of the master key sitting in the mobile bundle, and the upgrade paths (per-event provisioning, DESFire) that would close it without changing the reconciliation model.

7.3 Closing thought

The single sentence the thesis ends on is this: when the network is mostly there, treating offline as a bounded fallback rather than the foundation of the architecture turns out to be both simpler to build and easier to operate, as long as the chip carries enough state to keep the rare offline path safe. The trade is a smaller feature surface during outages in exchange for a much smaller feature surface to maintain during the rest of the time. For a festival, that is the right way around.

Bibliography

- [CP03] Patrick Y. K. Chau and Simpson Poon. Octopus: An E-cash payment system success story. *Communications of the ACM*, 46(9):129–133, 2003.
- [DHJ⁺07] Giuseppe DeCandia, Deniz Hastorun, Madan Jampani, Gunavardhan Kakulapati, Avinash Lakshman, Alex Pilchin, Swaminathan Sivasubramanian, Peter Voshall, and Werner Vogels. Dynamo: Amazon’s highly available key-value store. In *Proceedings of the 21st ACM Symposium on Operating Systems Principles (SOSP 2007)*, pages 205–220, 2007.
- [eFe16] eFestivals. Download festival scraps on site cashless only payment system. <https://www.efestivals.co.uk/news/16/160301h.shtml>, 2016. Online; accessed 2026.
- [Eur09] European Parliament and Council of the European Union. Directive 2009/110/EC on the taking up, pursuit and prudential supervision of the business of electronic money institutions. <https://eur-lex.europa.eu/eli/dir/2009/110/oj/eng>, 2009. Online; accessed 2026.
- [Eur15] European Parliament and Council of the European Union. Directive (eu) 2015/2366 on payment services in the internal market (PSD2). <https://eur-lex.europa.eu/eli/dir/2015/2366/oj/eng>, 2015. Online; accessed 2026.
- [Fow05] Martin Fowler. Event sourcing. <https://martinfowler.com/eaDev/EventSourcing.html>, 2005. Online; accessed 2026.
- [GdKGM⁺08] Flavio D. Garcia, Gerhard de Koning Gans, Ruben Muijers, Peter van Rossum, Roel Verdult, Ronny Wichers Schreur, and Bart Jacobs. Dismantling MIFARE classic. In *Proceedings of the 13th European Symposium on Research in Computer Security (ESORICS 2008)*, volume 5283 of LNCS, pages 97–114. Springer, 2008.

- [GL02] Seth Gilbert and Nancy Lynch. Brewer’s conjecture and the feasibility of consistent, available, partition-tolerant web services. *ACM SIGACT News*, 33(2):51–59, 2002.
- [KB17] Martin Kleppmann and Alastair R. Beresford. A conflict-free replicated JSON datatype. Technical Report 10, 2017.
- [KWvHM19] Martin Kleppmann, Adam Wiggins, Peter van Hardenberg, and Mark McGranaghan. Local-first software: you own your data, in spite of the cloud. In *Proceedings of ACM Onward! 2019*, pages 154–178, 2019.
- [MDAB10] Steven J. Murdoch, Saar Drimer, Ross Anderson, and Mike Bond. Chip and PIN is broken. In *Proceedings of the 2010 IEEE Symposium on Security and Privacy*, pages 433–446, 2010.
- [Ope26] OpenAI. Scaling PostgreSQL to power 800 million ChatGPT users. <https://openai.com/index/scaling-postgresql/>, 2026. Online; accessed April 2026.
- [Rie11] Eric Ries. *The Lean Startup*. Crown Business, New York, 2011.
- [SBH24] Lachlan Schomburgk, Alessandro Belli, and Arvid O. I. Hoffmann. Less cash, more splash? A meta-analysis on the cashless effect. *Journal of Retailing*, 100(3):382–403, 2024.
- [SCZ⁺16] Weisong Shi, Jie Cao, Quan Zhang, Youhuizi Li, and Lanyu Xu. Edge computing: Vision and challenges. *IEEE Internet of Things Journal*, 3(5):637–646, 2016.
- [SDK24] SDK.finance. Closed-loop payment systems: Definition, benefits, and business cases. <https://sdk.finance/blog/closed-loop-payment-systems-definition-benefits-and-business>, 2024. Online; accessed 2026.
- [Shi02] Yasutomo Shirakawa. JR East contactless IC card automatic fare collection system “Suica”. In *Proceedings of the 7th IEEE International Symposium on High Assurance Systems Engineering (HASE 2002)*, pages 3–10, 2002.
- [SJL⁺13] Muhammad Zubair Shafiq, Lusheng Ji, Alex X. Liu, Jeffrey Pang, Shobha Venkataraman, and Jia Wang. A first look at cellular network performance during crowded events. In *Proceedings of the ACM SIGMETRICS 2013*, pages 17–28, 2013.

- [Son24a] Sony Corporation. Case study: Hong Kong octopus card. <https://www.sony.co.jp/en/Products/felica/casestudy/article/octopus.html>, 2024. Online; accessed 2026.
- [Son24b] Sony Corporation. FeliCa – contactless IC card technology. <https://www.sony.net/Products/felica/about/>, 2024. Online; accessed 2026.
- [SPBZ11] Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. Conflict-free replicated data types. In *Proceedings of the 13th International Symposium on Stabilization, Safety, and Security of Distributed Systems (SSS 2011)*, pages 386–400, 2011.
- [Tap24] Tappit. RFID cashless payments for events. <https://tappit.com/pay/rfid>, 2024. Online; accessed 2026.
- [Vog09] Werner Vogels. Eventually consistent. *Communications of the ACM*, 52(1):40–44, 2009.
- [You10] Greg Young. CQRS documents. https://cQRS.files.wordpress.com/2010/11/cQRS_documents.pdf, 2010. Online; accessed 2026.